

Análisis de Requerimientos No Funcionales y Seguridad

Actividad N°: 7 **Fecha:** 09/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Definir en detalle los requerimientos no funcionales del sistema, con énfasis en seguridad, y establecer los mecanismos técnicos concretos que se usarán para cumplirlos.

2. Autenticación y autorización

Aspecto	Mecanismo definido
Autenticación	JSON Web Token (JWT), firmado con <code>JWT_SECRET</code> configurado por variable de entorno
Verificación de sesión	Middleware <code>auth</code> valida el token en cada request y recupera el usuario desde la base de datos, excluyendo el campo <code>password</code>
Usuarios inactivos	Un usuario con <code>activo: false</code> es rechazado aunque su token siga siendo técnicamente válido
Autorización por rol	Middleware <code>authorize(...roles)</code> bloquea el acceso a rutas según el rol del usuario autenticado (Jefe / Staff)
Contraseñas	Hasheadas con <code>bcryptjs</code> , salt de 12 rondas, nunca se devuelven en las respuestas JSON (<code>toJSON()</code> las elimina del objeto usuario)

3. Seguridad de la aplicación web

Mecanismo	Detalle
Helmet	Aplica cabeceras HTTP de seguridad por defecto (protección contra XSS, sniffing de MIME, clickjacking)
Rate limiting	Máximo 200 peticiones por minuto por IP; excluye rutas de <code>/health</code> y <code>check-changes</code> para no bloquear monitoreo ni sincronización legítima
CORS	Lista blanca explícita de orígenes permitidos en desarrollo (<code>localhost:3000</code> , <code>localhost:5173</code>) y en producción (dominios configurados + subdominios <code>*.onrender.com</code>); cualquier otro origen es rechazado
Validación de entrada	<code>express-validator</code> en el backend + validación de campos obligatorios en formularios del frontend
Manejo de errores	Middleware global que normaliza errores de validación, JWT y duplicados sin exponer detalles internos en producción

4. Seguridad de datos y auditoría

- Toda operación sensible (canje, impresión, reimpresión) genera un registro en **AuditLog** con usuario, IP y detalles — esto no es solo un requerimiento funcional, también es un control de seguridad (no repudio: un usuario no puede negar haber realizado una acción).
- Los datos de auditoría son de solo lectura desde la interfaz (no se exponen endpoints de edición/borrado de logs).
- El **Ticket ID** es único e indexado para impedir inconsistencias de datos entre reimportaciones.

5. Requerimientos No Funcionales (RNF) — versión final

ID	Categoría	Requerimiento
RNF-01	Seguridad	Autenticación JWT con expiración de sesión
RNF-02	Seguridad	Contraseñas con hash bcrypt (salt 12), nunca en texto plano ni en respuestas de API
RNF-03	Seguridad	Control de acceso por rol en cada endpoint sensible del backend
RNF-04	Seguridad	Cabeceras HTTP seguras (Helmet) y lista blanca de CORS
RNF-05	Seguridad	Límite de tasa de peticiones (rate limiting) para mitigar abuso/fuerza bruta
RNF-06	Disponibilidad	Soportar al menos 10 usuarios concurrentes sin degradación perceptible
RNF-07	Rendimiento	Búsquedas de boletos con tiempo de respuesta aceptable sobre miles de registros (índices en MongoDB: Ticket ID, Seat, updatedAt, compuestos)
RNF-08	Trazabilidad	Ninguna operación de canje/reimpresión debe perderse; auditoría inmutable de cada acción
RNF-09	Usabilidad	Interfaz operable con eficiencia bajo presión de tiempo (fila de personas esperando en el evento)
RNF-10	Portabilidad/Despliegue	Backend y frontend desplegables de forma independiente en la nube (Render), configurables por variables de entorno

6. Riesgos de seguridad identificados para mitigar en el diseño

- **Secretos en el código fuente:** se identifica como riesgo mantener credenciales de base de datos o secretos dentro de archivos versionados (ej. archivos de configuración de despliegue); deben manejarse siempre como variables de entorno gestionadas desde la plataforma de despliegue, nunca en texto plano en el repositorio.
- **Suplantación de punto de trabajo:** el filtro por punto de trabajo de un Staff debe forzarse en el backend (no confiar en el valor enviado desde el frontend).

- **Repetición de canje por condición de carrera:** dos solicitudes casi simultáneas para el mismo Ticket ID deben resolverse de forma que solo una tenga éxito.

7. Conclusiones del día

Se documentaron los mecanismos de seguridad ya implementados en el sistema (JWT, bcrypt, Helmet, rate limiting, CORS, RBAC, auditoría) y se formalizaron como requerimientos no funcionales, además de identificar riesgos a vigilar durante el desarrollo y despliegue.